



DOSSIER PROJET

Mohamed-Zouhair REGHAI

Développeur Web et Web Mobile

Promotion 2025-2026

Gestion Club

Table des matières

Avant-propos

1. La liste des compétences du référentiel couvertes par le projet	5
2. Le contexte du projet	
2.1 Présentation de l'association et du service	6
2.2 Cahier des charges.....	6
2.3 Contraintes du projet, les livrables attendus.....	7
2.4 Environnement humain et technique, les objectifs de qualité.....	8
3. Les réalisations front end :	
3.1 Présentation des maquettes – Desktop et Mobile	10
3.2 Schéma d'enchaînement des maquettes	13
3.3 Captures d'écran des interfaces – Desktop et Mobile	13
3.4 Extraits de code – Parties statiques	15
3.5 Extraits de code – Parties dynamiques	16
3.6 Les fenêtres modales	17
4. Les réalisations back end :	
4.1 Présentation de la base de données	20
4.2 Extraits de code – Composants métier	25
4.3 Extraits de code – Composants d'accès aux données	29
5. Les éléments de sécurité de l'application	32
6. Les jeux d'essai	34
7. La veille	37
8. Annexes	
8.1 Maquettes des interfaces utilisateur	38
8.2 Interfaces utilisateur et le code correspondant	38
8.3 Code des composants métier les plus significatifs	40
8.4 Code des composants d'accès aux données les plus significatifs	40

Avant-propos — Méthodologie de travail

Le développement de MNS FC a été mené en autonomie sur une période d'environ 8 mois (février à septembre 2026), organisée en phases successives avec des objectifs intermédiaires fixés dès le départ, plutôt qu'en une seule phase de développement continue.

Le planning a suivi 5 grandes phases successives : la conception et la mise en place de la base de données, architecture, MCD/MLD (février à avril 2026), le maquettage et l'intégration HTML/CSS (mai à juin), le développement back-end PHP (juillet à août), la couche JavaScript (août à septembre), puis la rédaction des dossiers (septembre). Chaque objectif intermédiaire a été suivi via un outil de gestion de projet (Jira, tickets SCRUM), permettant de garder une visibilité réelle sur l'avancement plutôt qu'une estimation approximative.

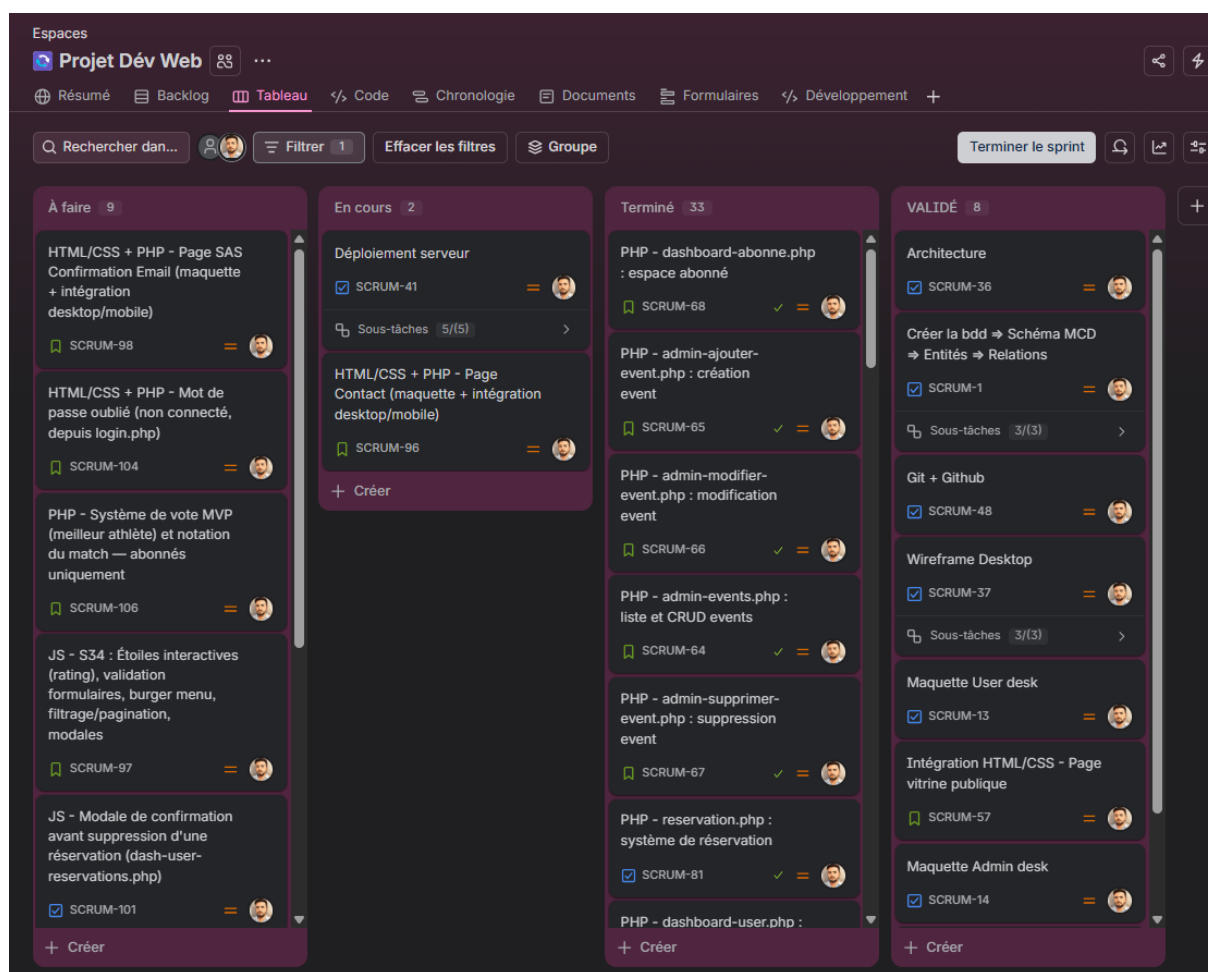


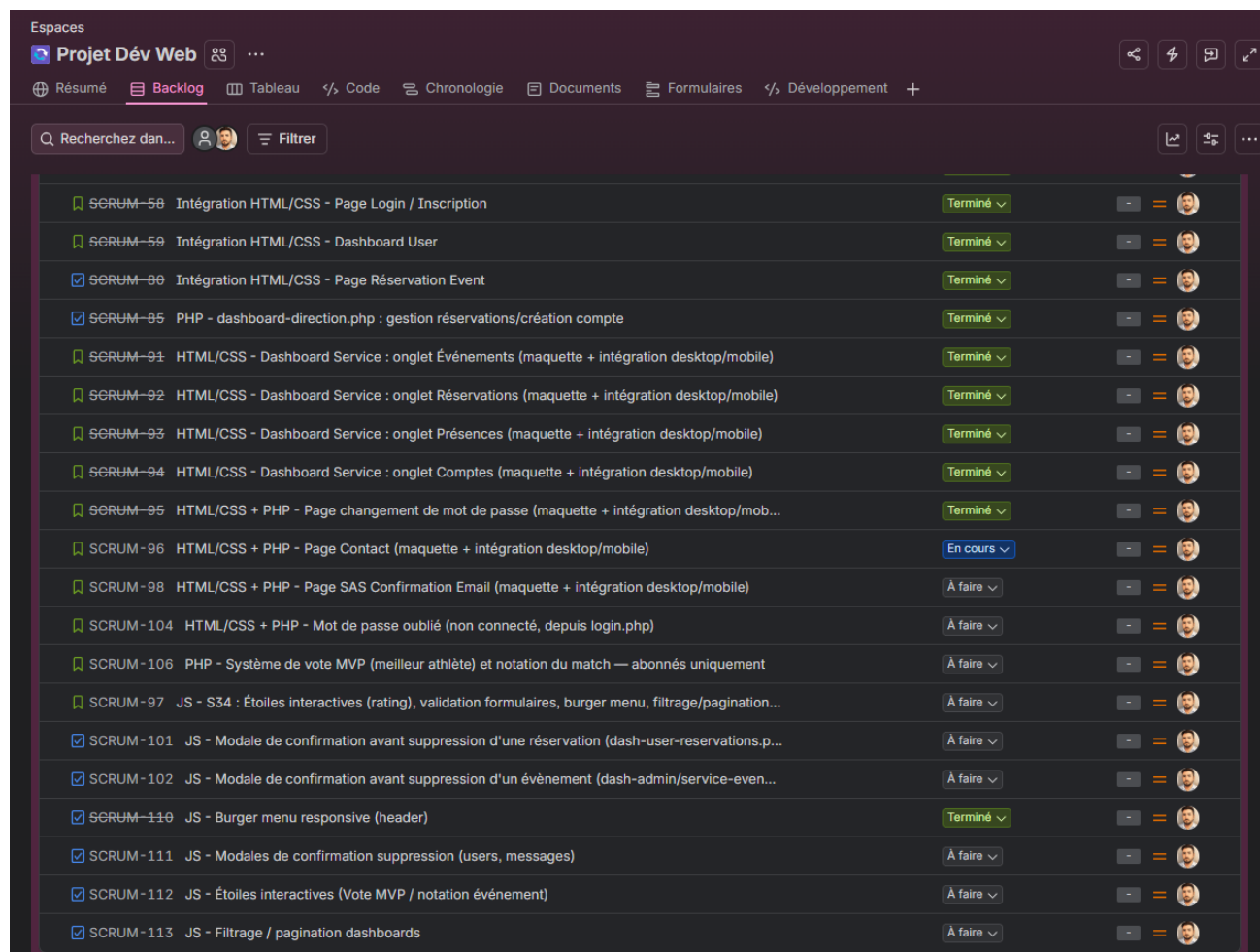
Figure 0.1 — Tableau Kanban du suivi de sprint (Jira)

Une colonne complémentaire (Validé) distingue les tickets ayant fait l'objet d'une validation formelle avant clôture définitive, garantissant une double vérification au-delà du simple achèvement technique.

Cette organisation par sprints a permis d'identifier tôt certains écarts entre le planning prévisionnel et la réalité du développement — par exemple, une semaine de congés en plein milieu du sprint back-end a décalé d'autant l'ensemble des étapes suivantes.

Un principe de qualité a été retenu et assumé tout au long du projet : maîtriser **60 à 70%** du périmètre fonctionnel du cahier des charges à un niveau abouti, plutôt que de livrer l'intégralité des fonctionnalités de façon superficielle.

Les fonctionnalités volontairement repoussées (gestion des présences par QR code, vote MVP côté back-end, mot de passe oublié) sont documentées comme telles dans ce dossier, avec leur justification, plutôt que passées sous silence.



The screenshot shows a Jira Backlog for the project 'Projet Dev Web'. The interface includes a top navigation bar with tabs for 'Résumé', 'Backlog', 'Tableau', 'Code', 'Chronologie', 'Documents', 'Formulaires', and 'Développement'. Below the navigation bar is a search bar and a filter button. The main content area displays a list of tasks, each with a checkbox, a task ID, a description, a status dropdown, a priority icon, and an assignee icon.

ID	Description	Status	Assignee
SCRUM-58	Intégration HTML/CSS - Page Login / Inscription	Terminé	[Assignee]
SCRUM-59	Intégration HTML/CSS - Dashboard User	Terminé	[Assignee]
SCRUM-80	Intégration HTML/CSS - Page Réservation Event	Terminé	[Assignee]
SCRUM-85	PHP - dashboard-direction.php : gestion réservations/création compte	Terminé	[Assignee]
SCRUM-91	HTML/CSS - Dashboard Service : onglet Événements (maquette + intégration desktop/mobile)	Terminé	[Assignee]
SCRUM-92	HTML/CSS - Dashboard Service : onglet Réservations (maquette + intégration desktop/mobile)	Terminé	[Assignee]
SCRUM-93	HTML/CSS - Dashboard Service : onglet Présences (maquette + intégration desktop/mobile)	Terminé	[Assignee]
SCRUM-94	HTML/CSS - Dashboard Service : onglet Comptes (maquette + intégration desktop/mobile)	Terminé	[Assignee]
SCRUM-95	HTML/CSS + PHP - Page changement de mot de passe (maquette + intégration desktop/mobile)	Terminé	[Assignee]
SCRUM-96	HTML/CSS + PHP - Page Contact (maquette + intégration desktop/mobile)	En cours	[Assignee]
SCRUM-98	HTML/CSS + PHP - Page SAS Confirmation Email (maquette + intégration desktop/mobile)	À faire	[Assignee]
SCRUM-104	HTML/CSS + PHP - Mot de passe oublié (non connecté, depuis login.php)	À faire	[Assignee]
SCRUM-106	PHP - Système de vote MVP (meilleur athlète) et notation du match — abonnés uniquement	À faire	[Assignee]
SCRUM-97	JS - S34 : Étoiles interactives (rating), validation formulaires, burger menu, filtrage/pagination...	À faire	[Assignee]
SCRUM-101	JS - Modale de confirmation avant suppression d'une réservation (dash-user-reservations.p...	À faire	[Assignee]
SCRUM-102	JS - Modale de confirmation avant suppression d'un événement (dash-admin/service-even...	À faire	[Assignee]
SCRUM-110	JS - Burger menu responsive (header)	Terminé	[Assignee]
SCRUM-111	JS - Modales de confirmation suppression (users, messages)	À faire	[Assignee]
SCRUM-112	JS - Étoiles interactives (Vote MVP / notation événement)	À faire	[Assignee]
SCRUM-113	JS - Filtrage / pagination dashboards	À faire	[Assignee]

1. Liste des compétences du référentiel couvertes par le projet

Le projet Gestion de club couvre les compétences professionnelles suivantes du référentiel DWWM :

N°	Activités Types	N°	Compétences professionnelles
1	Front End	1	Installer et configurer son environnement de travail
		2	Maquetter des interfaces utilisateur web ou web mobile
		3	Réaliser des interfaces utilisateur statiques web et web mobile
		4	Développer la partie dynamique des interfaces utilisateur web et web mobile
2	Back End	1	Mettre en place une base de données relationnelle
		2	Développer des composants d'accès aux données SQL
		3	Développer des composants métier côté serveur
		4	Documenter le déploiement d'une application dynamique web ou web mobile

Tableau 1.1 - Les huit compétences du référentiel DWWM.

Le tableau ci-dessus reprend les 8 compétences professionnelles qui se regroupent dans deux aspects de spécialisation, le front-end et le back-end. Les deux ont été traités de la même importance mais pas avec le même enchaînement. J'ai commencé le travail d'abord avec la réflexion puis l'architecture de la base de données, puis se sont enchaînées les étapes de maquettage et d'intégration pour revenir par la suite au développement des composants d'accès aux données et des composants métier. J'ai fini alors par développer la partie dynamique des interfaces et la documentation.

2. Contexte du projet :

2.1 - Présentation de l'entreprise et du service

Le projet MNS FC a été réalisé dans le cadre de la formation DWWM dispensée par Metz Numeric School. Ce projet proposé parmi une liste de 8 autres par l'école répond à un besoin réel dans ma vie personnelle exprimé par mon Club de football l'Athletic Club Augny (ACA).

Le Club ACA organise régulièrement des événements sportifs (matches, tournois, stages) et d'autres de loisirs (fête, bingo, etc.) pour les adhérents et leurs familles.

Jusqu'à présent, la gestion des inscriptions et des réservations à ces événements était réalisée manuellement avec plusieurs personnes en charge selon leurs présences avec tous les problèmes que cela peut apporter comme suivi, modification, annulation et bien sûr la gestion des paiements.

Les échanges se font par e-mail, sms ou téléphone, ce qui entraînent des pertes d'information, des doublons et une charge administrative importante. Gest Club est une application web développée pour numériser, automatiser et sécuriser ce processus de gestion.

2.2 - Cahier des charges

Les besoins fonctionnels :

L'application doit permettre :

- Aux adhérents de MNS FC : S'inscrire, se connecter, consulter les événements disponibles et effectuer une réservation (dans la limite de 2 places par événement)
- Aux administrateurs : Gérer les événements (création, modification, suppression), gérer les adhérents, consulter et gérer les réservations et enfin suivre et tracer les présences par événement le jour j.
- Au service réservation : Accéder aux mêmes fonctionnalités de gestion des événements, des réservations et des présences que l'administrateur, à l'exception de la gestion des comptes adhérents
- Une authentification sécurisée avec gestion de 3 rôles distincts (adhérent, administrateur, service réservation)
- Une interface responsive, adaptée aussi bien aux écrans desktop qu'aux mobile

Les besoins non fonctionnels :

- Sécurité : protection contre les injections SQL (requêtes préparées PDO), protection contre les failles XSS, gestion sécurisée des sessions et des mots de passe
- Ergonomie : navigation intuitive, charte graphique cohérente sur l'ensemble des pages
- Performance : chargement rapide des pages, requêtes SQL optimisées

- Maintenabilité : code structuré et commenté, versionné sur GitHub

Standard et qualité :

L'application doit respecter les standards actuels notamment :

- Compatibilité avec les navigateurs courant du marché
- Respect des standards du Web (HTML, CSS) dans les versions les plus à jour sous réserve du respect du point précédent
- Respect du Règlement Générale sur la Protection des Données (RGPD)
- Anonymisation et cryptage des données sensibles
- Respect des principes d'éco-conception

De manière générale, tout standard permettant d'améliorer la qualité du projet tant sur l'organisation que la production.

2.3 - Contraintes du projet et livrables attendus

Les contraintes :

- Stack : PHP procédural + MySQL + HTML/CSS/JavaScript (J'ai fait le choix de n'utiliser aucun framework front-end ou back-end et aucune programmation orientée objet dans le cadre de ce projet fil rouge = PFR)
- Maquettage, j'ai choisi de travailler sur Figma avant toute intégration
- Environnement de développement : Docker en local (PHP/Apache, MySQL :8.0, PhpMyAdmin)
- Déploiement : transfert vers la VM de l'école via FileZilla, puis mise en place d'un déploiement continu vers un hébergement de production (Hostinger) déclenché à chaque push sur le dépôt GitHub
- Versioning obligatoire sur un dépôt GitHub public
- Soutenance en octobre 2026 devant un jury professionnel

Les livrables attendus :

Les livrables sont cadencés par un calendrier prévisionnel à quatre phases, chacune associée à une partie du dossier projet :

- Application web MNS FC fonctionnelle
- Code source versionné sur GitHub
- Dossier Projet complet (numérique + papier)

- Dossier Professionnel (numérique + papier)
- Maquettes Figma (desktop et mobile)
- Schéma de base de données (MCD, MLD, script SQL)

Comparaison avec le planning prévisionnel de l'école

Metz Numeric School fournit en début de formation un planning prévisionnel indicatif, en 4 phases réparties de mars à juillet. Le déroulement réel du projet a suivi les mêmes grandes étapes, dans un ordre et sur une durée légèrement adaptée à la réalité du projet : démarrage effectif en février plutôt qu'en mars, et surtout traitement de la base de données en amont du maquetage, plutôt qu'après, afin de sécuriser le modèle de données avant de concevoir les interfaces qui s'appuient dessus. La durée totale s'est également étendue de 5 à 8 mois, pour approfondir chaque phase plutôt que respecter un calendrier serré.

Phase prévisionnelle (école)	Contenu prévu	Réalisation effective
Phase 1 (mars)	CDC, charte graphique, maquettes Figma, dépôt Git, intégration front-end	Réalisé, décalé sur mai-juin (après la BDD)
Phase 2 (avril)	Conception de la base de données, choix technologique	Réalisé en premier, dès février-avril
Phase 3 (juin)	Dépôt du code, 30% des fonctionnalités, tests, sécurisation	Réalisé, sur juillet-août (développement back-end)
Phase 4 (juillet)	Déploiement, documentation, axes d'amélioration, finalisation	Réalisé, étendu à août-septembre (JS puis rédaction)

Tableau 2.1 - Comparaison entre le planning prévisionnel et le déroulement réel du projet.

2.4 - L'environnement humain et technique, les objectifs de qualité

L'environnement humain :

- Développeur : REGHAI Mohamed-Zouhair, stagiaire DWWM, promotion 2025-2026
- Référent pédagogique : QUIRIN Franck, formateur Metz Numeric School
- Jury de soutenance : Jury professionnel externe, octobre 2026

L'environnement technique :

- Langages : PHP 8.2 (procédural), HTML5, CSS3, JavaScript (ES6)
- Base de données : MySQL 8.0, administrée via PhpMyAdmin

- Environnement local : Docker (PHP/Apache, MySQL, PhpMyAdmin)
- Déploiement : VM de l'école (transfert via Filezilla) et hébergement de production Hostinger (déploiement continu via Git)
- Maquettage : Figma (maquettes desktop et mobile validées)
- Architecture BDD : Looping
- Versioning : Git / GitHub
- Gestion de projet : Jira (tickets SCRUM)
- IDE : Visual Studio Code
- Charte graphique : 2 couleurs principales (bleu nuit #1a1a2e, orange #ff6b35) et 7 couleurs secondaires/fonctionnelles (clair, beige, rouge, vert, gris, sombre, marine) ; typographie Montserrat (titres) et Roboto (texte courant)

Les objectifs qualité :

- Code lisible, indenté et commenté
- Responsive-design : conception mobile, avec breakpoints à 768px, 992 et 1200px
- Sécurisation systématique des entrées utilisateur, côté client comme côté serveur
- Maîtrise de 60 à 70% du périmètre fonctionnel à un niveau de qualité élevé, plutôt qu'un périmètre complet mais bâclé
- Application utilisable comme portfolio professionnel après la soutenance

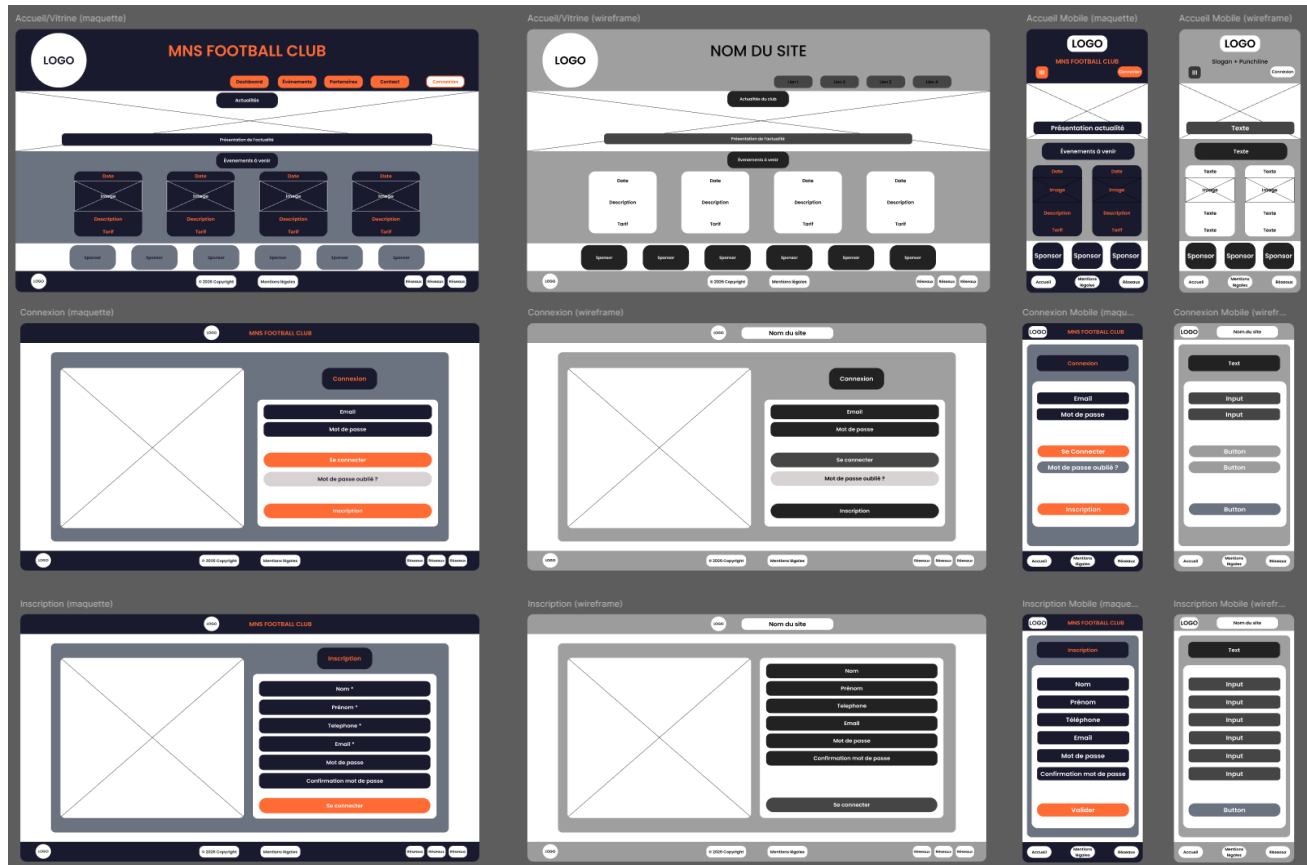
3. Réalisations front end :

Le développement front-end de MNS FC n'a pas été une ligne droite. L'approche retenue dès le départ est le mobile-first, avec des breakpoints progressifs à 768px, 992px et 1200px, le travail a nécessité plusieurs itérations sur le dashboard et le menu burger mobile, la partie la plus délicate de l'intégration.

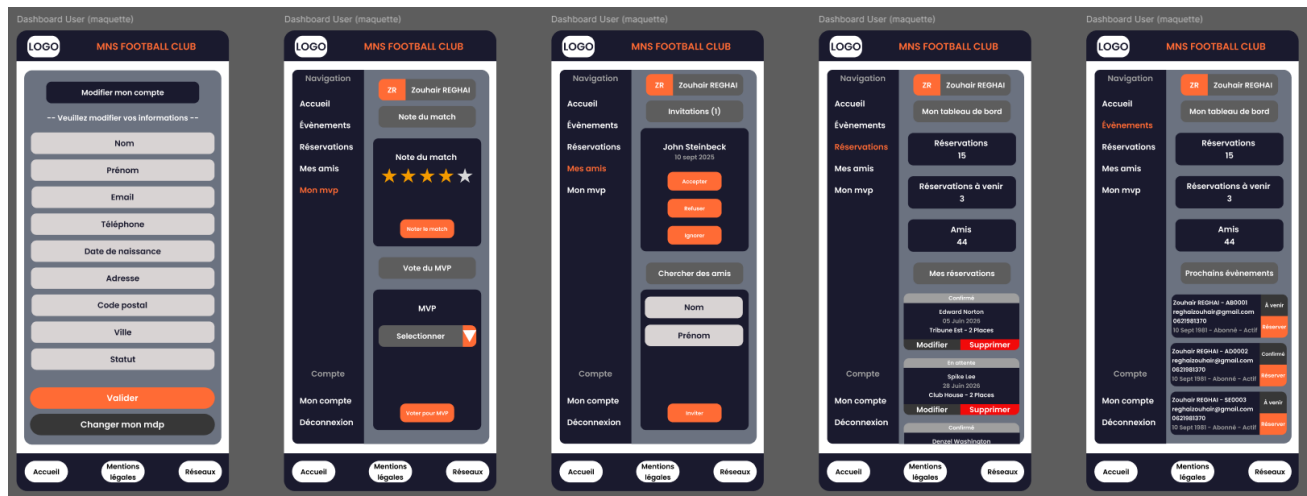
Une première version du menu, animée en CSS (transition sur la propriété `left` pour simuler un glissement latéral), a révélé un bug subtil : les styles propres à l'état « ouvert » du menu (couleur de fond, disposition verticale des liens) n'étaient appliqués que via une classe (`.open`), ce qui provoquait un changement brutal de l'apparence du menu pendant les 30 ms de fermeture, alors que sa position continuait de glisser. La correction a consisté à séparer clairement ce qui doit rester permanent (*l'apparence du panneau*) de ce qui doit réellement varier (*sa position*), une distinction qui s'est révélée être un principe réutilisable pour toute animation d'interface par la suite.

Côté validation des formulaires, le choix a été fait de générer dynamiquement les messages d'erreur en JavaScript (via `createElement`) plutôt que de les figer dans le HTML, conformément à l'enseignement reçu en formation sur ce point. Cette approche a demandé une gestion rigoureuse pour éviter la duplication des messages à chaque nouvelle tentative de saisie invalide (vérification systématique de l'existence de l'élément avant sa création).

3.1 - Présentation des maquettes – desktop et mobile



Les maquettes mobiles :



Les maquettes desktop :



3.2 - Schéma d'enchaînement des maquettes

Le schéma ci-dessous présente l'enchaînement des écrans de l'application, depuis l'arrivée d'un visiteur jusqu'à son accès aux 3 Dashboard selon son profil (adhérent, service réservation, administrateur).

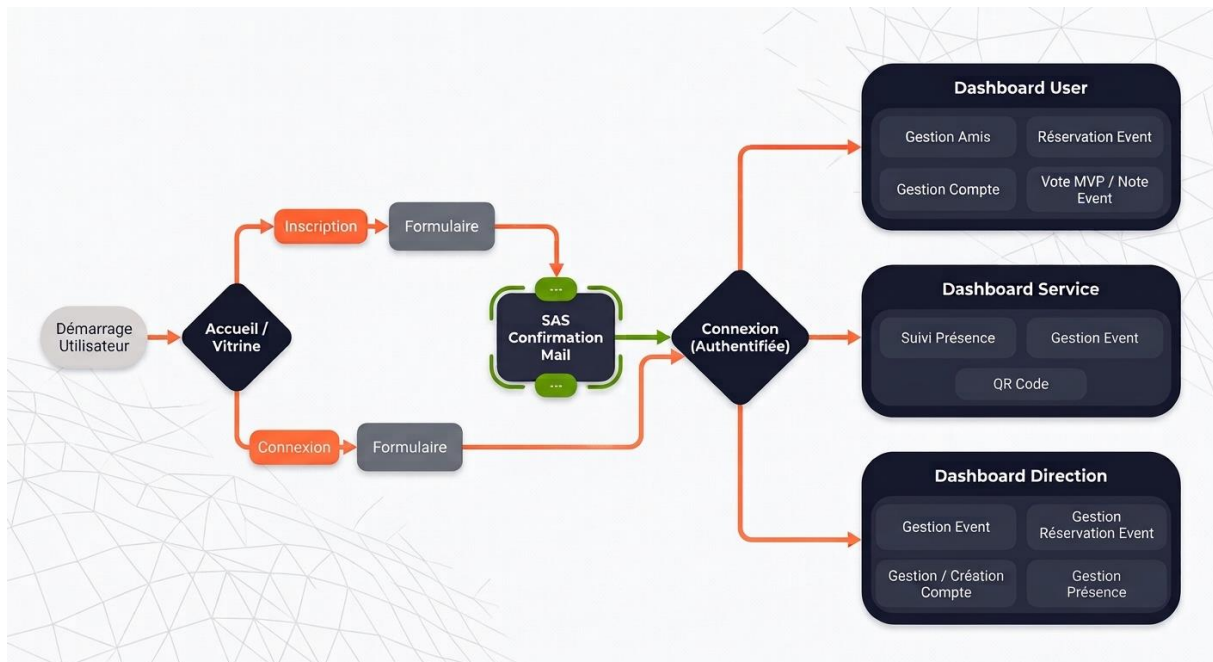


Figure 3.1 — Schéma d'enchaînement des maquettes MNS FC

3.3 - Captures d'écran des interfaces – Desktop et Mobile



3.4 - Extraits de code – Parties statiques

L'ensemble de la charte graphique repose sur des variables CSS centralisées, permettant de modifier une couleur ou un espacement en un seul endroit pour toute l'application :

```
:root {  
  /* Les couleurs */  
  --color-primary: #1a1a2e;  
  --color-secondary: #ff6b35;  
  --color-light: #ebebeb;  
  --color-beige: #d8d3d3;  
  --color-gray: #6b7280;  
  --color-dark: #373737;  
  --color-red: #c80000;  
  --color-green: #609f08;  
  
  /* Les textes */  
  --font-heading: 'Montserrat', sans-serif;  
  --font-body: 'Roboto', sans-serif;  
  
  /* Les borders */  
  --border-radius-s: 5px;  
  --border-radius-m: 10px;  
  --border-radius-l: 15px;  
  --border-radius-xl: 25px;  
  
  /* Les borders */  
  --border-radius-s: 5px;  
  --border-radius-m: 10px;  
  --border-radius-l: 15px;  
  --border-radius-xl: 25px;  
  
  /* Les padding */  
  --padding-s: 5px;  
  --padding-m: 10px;  
  --padding-l: 20px;  
  --padding-xl: 30px;  
  --padding-xxl: 50px;  
  
  /* Les margins */  
  --margin-s: 5px;  
  --margin-m: 10px;  
  --margin-l: 20px;  
  --margin-xl: 30px;  
  --margin-xxl: 50px;  
}
```

Figure 3.5 — Extrait code variables.css

Le menu de navigation (header.php), commun à toutes les pages, illustre la structure responsive-first : le bouton burger n'est visible qu'en dessous de 768px, la navigation complète étant affichée nativement au-delà.

```
<nav>  
  <ul>  
    <li>  
      <?php  
        if ($profil === 'administrateur' || $profil === 'service') { ?>  
          <a href="staff-events.php">Dashboard</a>  
        <?php } elseif ($profil === 'abonne') { ?>  
          <a href="user-events.php">Mon Espace</a>  
        <?php } else { ?>  
          <a href="index.php">Accueil</a>  
        <?php } ?>  
    </li>  
    <li><a href="../public/events.php">Évènements</a></li>  
    <li><a href="#">Partenaires</a></li>  
    <li><a href="#">Contact</a></li>  
  </ul>  
</nav>
```

Figure 3.6 — Extrait code header.php

Le comportement responsif du menu est entièrement piloté par media query, en confinant la position et l'animation du panneau mobile à un seul bloc pour éviter toute fuite de style vers l'affichage desktop :

```

/* Format mobile */
@media screen and (max-width: 767px) {
  header .bottom nav {
    display: block;
    position: absolute;
    top: 55px;
    left: -450px;
    transition: left 0.3s;
    z-index: 10;
  }

  header .bottom nav.nav-open {
    left: -5px;
  }
}

```

Figure 3.7 — Extrait code header.css

3.5 - Extraits de code – Parties dynamiques

Le menu burger repose sur un simple basculement de classe CSS, associé au changement d'icône (menu / croix) :

```

const burger = document.getElementById('burger-btn');
const nav = document.querySelector('nav');
const i = burger.querySelector('i');

burger.addEventListener('click', () => {
  nav.classList.toggle('nav-open');
  i.classList.toggle('bx-menu');
  i.classList.toggle('bx-x');
})

```

Figure 3.8 — Extrait code header.js

La validation des formulaires suit un pattern homogène sur l'ensemble du projet : un message d'erreur est créé dynamiquement s'il n'existe pas déjà (évitant toute duplication à chaque nouvelle tentative), puis retiré dès que la saisie redevient valide. Extrait de la validation de l'email sur inscription.js :

```

email.addEventListener('change', (event) => {
  const emailError = document.getElementById('emailError');
  if (email.value.includes('@') && emailError !== null) {
    emailError.remove()
  }
})

```

Figure 3.9 — Extrait code inscription.js

Enfin, la complexité du mot de passe est vérifiée en temps réel au moyen d'expressions régulières, avec un retour visuel immédiat sur chaque critère :

```
//Vérification du mot de passe
const pwdLength = document.getElementById('pwd-criteria-length');
const pwdSpecial = document.getElementById('pwd-criteria-special');
const pwdUppercase = document.getElementById('pwd-criteria-uppercase');
const pwdNumeric = document.getElementById('pwd-criteria-numeric');

newPwd.addEventListener('keyup', () => {
  // 8 caractères minimum :
  if (newPwd.value.length >= 8) {
    |   pwdLength.classList.add('success');
  } else {
    |   pwdLength.classList.remove('success');
  }

  // caractère spécial minimum :
  let regExSpecial = /[#@!?$%&]/;
  if (regExSpecial.test(newPwd.value)) {
    |   pwdSpecial.classList.add('success');
  } else {
    |   pwdSpecial.classList.remove('success');
  }

  // caractère majuscule minimum :
  let regExUpper = /[A-Z]/;
  if (regExUpper.test(newPwd.value)) {
    |   pwdUppercase.classList.add('success');
  } else {
    |   pwdUppercase.classList.remove('success');
  }

  // caractère numérique minimum :
  let regExNumeric = /[0-9]/;
  if (regExNumeric.test(newPwd.value)) {
    |   pwdNumeric.classList.add('success');
  } else {
    |   pwdNumeric.classList.remove('success');
  }
})
```

Figure 3.10 — Extrait code password.js

3.6 - Les fenêtres modales : confirmation et alerte

Sur MNS FC, toute action de création, modification ou suppression passe par une fenêtre modale plutôt qu'un simple rechargement de page. Deux problèmes distincts sont visés : qu'une action difficile à annuler parte d'un simple clic sans que l'utilisateur ait le temps de se rétracter, et qu'un utilisateur reste dans le doute après avoir soumis un formulaire, sans savoir si son action a réellement abouti.

Deux natures de modales coexistent, avec un rôle différent chacune :

- **Les modales de confirmation**, affichées avant l'action, sur les opérations difficiles à annuler (suppression d'un événement, d'un adhérent, d'une réservation). La requête réelle n'est déclenchée qu'après un clic explicite sur "Confirmer" dans la modale, jamais directement au premier clic sur le bouton ou le lien.
- **Les modales d'alerte**, affichées après l'action, pour confirmer un succès (réservation enregistrée, mot de passe modifié) ou signaler un échec (erreur serveur, champ invalide), avant que l'utilisateur ne reprenne la main sur la page.

Ce chantier corrige au passage un point faible identifié pendant la rédaction de ce dossier : la suppression d'un événement (*staff-delete-event.php*) s'exécutait jusque-là au chargement d'une simple URL en GET, sans aucune confirmation, un lien suffisait à déclencher la suppression. Le passage par une modale de confirmation, couplé au passage de la requête en POST, ferme ce point (voir 4.3).

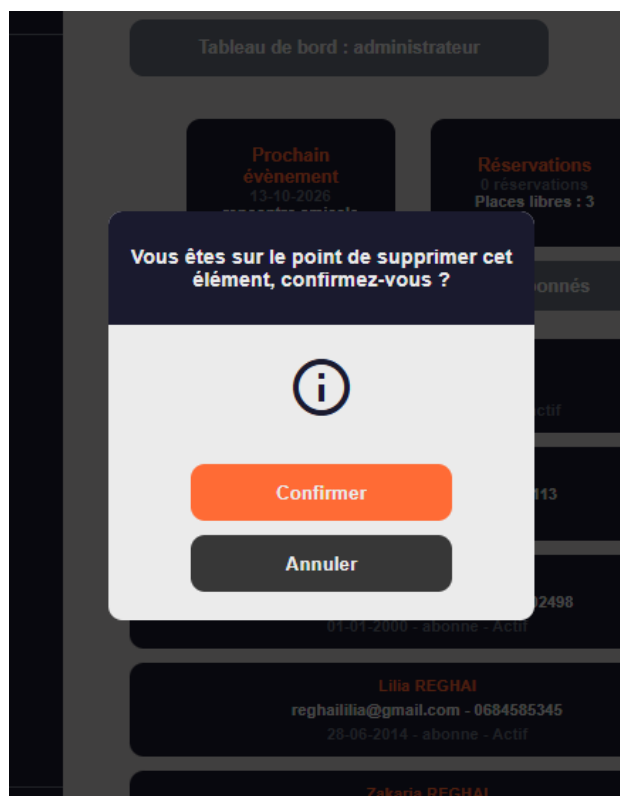


Figure 3.11 — Modale de confirmation avant suppression

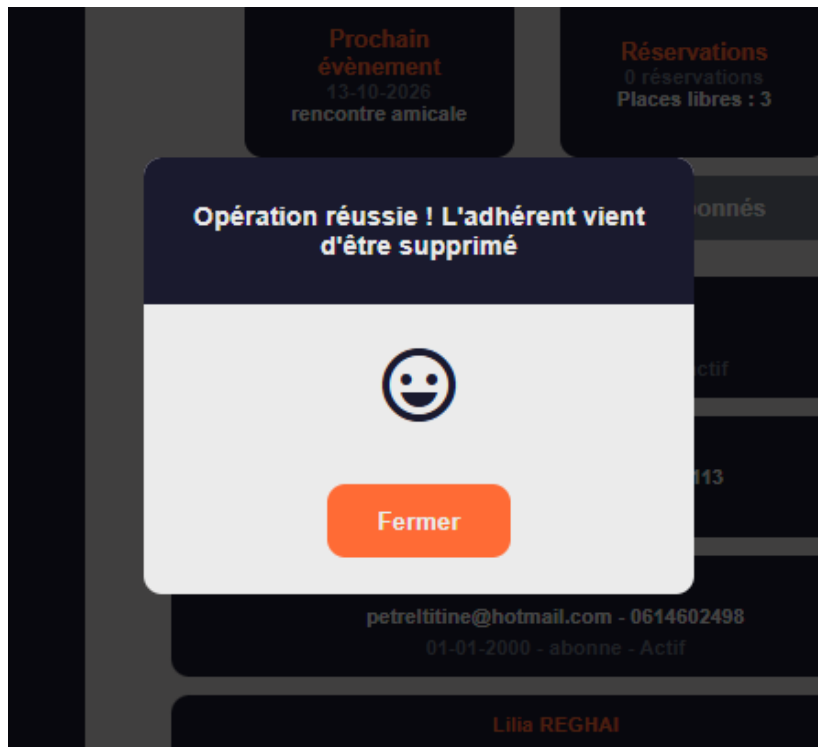


Figure 3.12 — Modale d'information succès

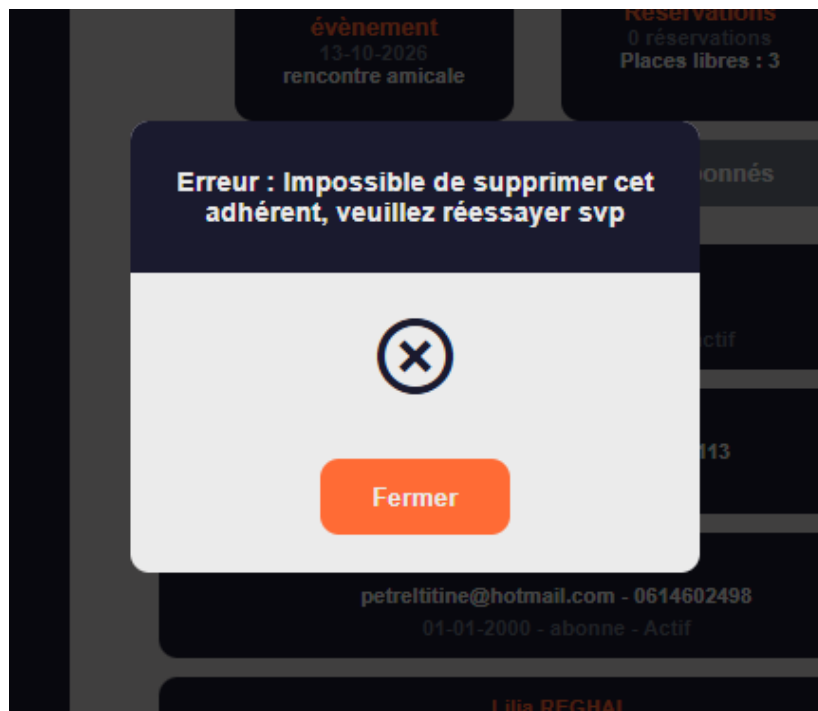


Figure 3.13 — Modale d'information échec

4. Réalisations back end :

La base de données a fait l'objet d'une relecture approfondie en toute fin de projet, à l'occasion de la rédaction de ce dossier : documenter précisément le Modèle Logique de Données a mis en évidence que plusieurs relations pourtant utilisées dans le code au quotidien (orders vers events et users, events vers categories) n'étaient pas réellement contraintes au niveau du SGBD.

Plutôt que de simplement noter ce constat comme un axe d'amélioration parmi d'autres, j'ai fait le choix de corriger réellement la base de données en production, en réfléchissant relation par relation au comportement souhaité en cas de suppression (RESTRICT pour les réservations et les événements, afin d'éviter toute perte silencieuse de données ; CASCADE pour un vote MVP rattaché à un athlète qui n'a finalement pas joué, où conserver le vote n'aurait aucun sens ; SET NULL pour un message de contact, afin de préserver l'historique même après suppression du compte de l'expéditeur). En effectuant cette tâche j'ai dû faire face à une autre problématique qui est la présence de données orphelines présentes naturellement lors d'une modification d'une base peuplée de données de tests.

4.1 - Présentation de la base de données

Modèle Conceptuel de Données (MCD)

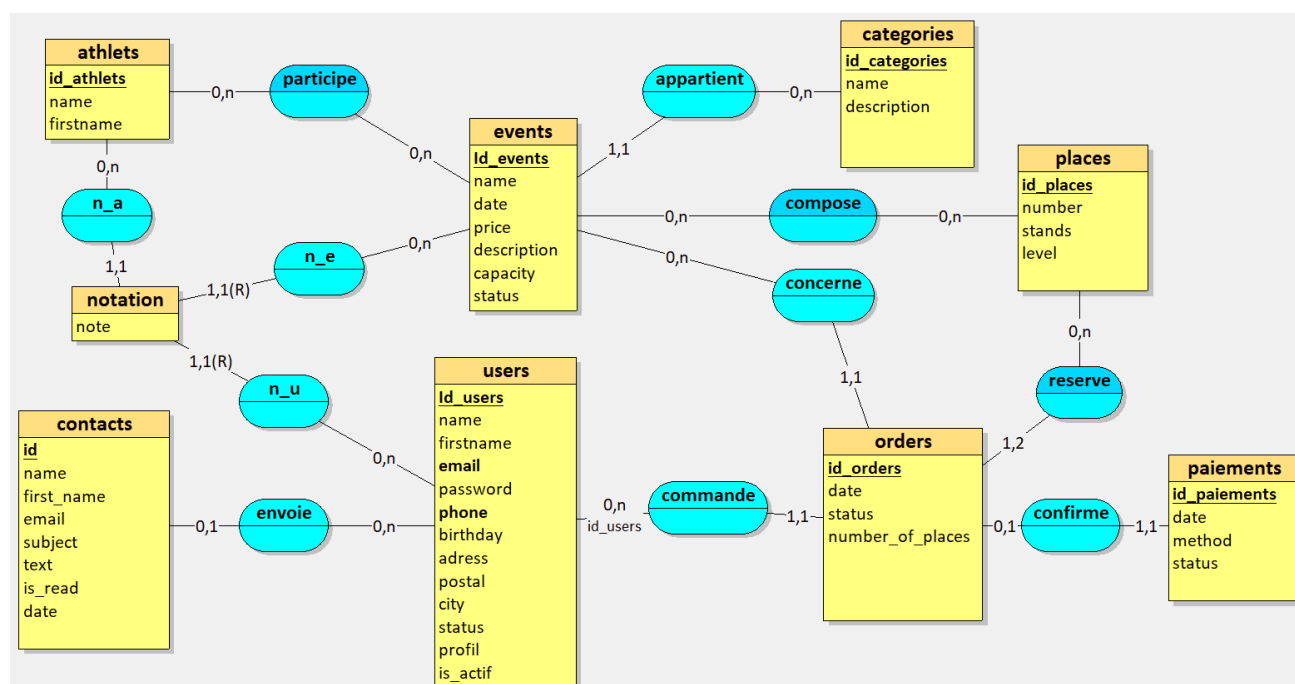


Figure 4.1 — Modèle conceptuel de données (MCD) MNS FC

La base de données `GestClub` (nom de code originel du projet) s'articule autour de 9 entités, reflétant les acteurs et objets métiers identifiés dans le cahier des charges de MNS FC :

- Relations principales : un event appartient à une category ; un user peut passer plusieurs orders et un event peut recevoir plusieurs orders (relation many-to-many résolue par la table orders, qui porte en plus le nombre de places réservées) ; un order peut donner lieu à un payment (relation 1-1) ; un user peut noter plusieurs events et proposer un athlète comme meilleur joueur via la table de liaison notation (clé primaire composite events_id + users_id : un seul vote par adhérent et par événement) ; un message de contact peut être rattaché à un user si l'expéditeur est connecté, ou rester anonyme (users_id à NULL) s'il s'agit d'un simple visiteur.

```

    erDiagram
        athletes ||--o{ participate : "participe"
        athletes ||--o{ events : "Id_events"
        athletes ||--o{ notation : "Id_users"
        athletes ||--o{ notation : "Id_events"
        athletes ||--o{ notation : "id_athlets"
        participate ||--o{ events : "Id_events"
        participate ||--o{ events : "id_athlets"
        events ||--o{ categories : "id_categories"
        events ||--o{ compose : "Id_events"
        events ||--o{ compose : "id_places"
        notation ||--o{ events : "Id_users"
        notation ||--o{ events : "Id_events"
        notation ||--o{ events : "id_athlets"
        contact ||--o{ users : "Id_users"
        contact ||--o{ users : "email"
        users ||--o{ orders : "Id_events"
        users ||--o{ orders : "id_users"
        orders ||--o{ places : "id_places"
        orders ||--o{ reserve : "id_orders"
        orders ||--o{ reserve : "id_places"
        paiements ||--o{ orders : "id_orders"
  
```

The diagram illustrates the following tables and their attributes:

- athlets**: id_athlets, name, firstname
- participe**: Id_events, id_athlets
- events**: Id_events, name, date_, price, description, capacity, status, id_categories
- categories**: id_categories, name, description
- compose**: Id_events, id_places
- notation**: Id_users, Id_events, note, id_athlets
- contact**: Id, name, first_name, email, subject, text, is_read, date_, Id_users
- users**: Id_users, name, first_name, email, password, telephone, birthday, adress, postal, city, profil, is_public
- orders**: id_orders, date_, status, number_of_places, Id_events, id_users
- places**: id_places, number, stands, level
- reserve**: id_orders, id_places
- paiements**: id_paiements, date_, method_, status, id_orders

21

```
-- Structure de la table `users`
--

CREATE TABLE `users` (
  `id` int NOT NULL,
  `name` varchar(100) CHARACTER SET utf8mb4 COLLATE utf8mb4_unicode_ci NOT NULL,
  `first_name` varchar(100) CHARACTER SET utf8mb4 COLLATE utf8mb4_unicode_ci NOT NULL,
  `email` varchar(100) CHARACTER SET utf8mb4 COLLATE utf8mb4_unicode_ci NOT NULL,
  `password` varchar(100) CHARACTER SET utf8mb4 COLLATE utf8mb4_unicode_ci NOT NULL,
  `phone` varchar(50) CHARACTER SET utf8mb4 COLLATE utf8mb4_unicode_ci NOT NULL,
  `birthday` date NOT NULL,
  `adress` varchar(255) CHARACTER SET utf8mb4 COLLATE utf8mb4_unicode_ci NOT NULL,
  `postal` varchar(10) CHARACTER SET utf8mb4 COLLATE utf8mb4_unicode_ci NOT NULL,
  `city` varchar(50) CHARACTER SET utf8mb4 COLLATE utf8mb4_unicode_ci NOT NULL,
  `status` enum('Fermé', 'Privé', 'Public') CHARACTER SET utf8mb4 COLLATE utf8mb4_unicode_ci NOT NULL DEFAULT 'Public',
  `profil` enum('administrateur', 'service', 'abonne') CHARACTER SET utf8mb4 COLLATE utf8mb4_unicode_ci NOT NULL DEFAULT 'abonne',
  `is_actif` tinyint(1) NOT NULL DEFAULT '1'
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_unicode_ci;

-- Structure de la table `events`
--

CREATE TABLE `events` (
  `id` int NOT NULL,
  `name` varchar(100) CHARACTER SET utf8mb4 COLLATE utf8mb4_unicode_ci NOT NULL,
  `description` text CHARACTER SET utf8mb4 COLLATE utf8mb4_unicode_ci NOT NULL,
  `image` varchar(255) CHARACTER SET utf8mb4 COLLATE utf8mb4_unicode_ci NOT NULL DEFAULT '../assets/images/events/default.jpg',
  `date` date NOT NULL,
  `price` decimal(10,0) NOT NULL,
  `scene` enum('Tribune Nord', 'Tribune Sud', 'Tribune Est', 'Tribune Ouest', 'Club House') CHARACTER SET utf8mb4 COLLATE utf8mb4_unicode_ci NOT NULL DEFAULT 'Tribune Sud',
  `capacity` int NOT NULL,
  `status` enum('À venir', 'Confirmé', 'Reporté', 'Annulé') CHARACTER SET utf8mb4 COLLATE utf8mb4_unicode_ci NOT NULL,
  `categories_id` int NOT NULL
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_unicode_ci;

-- Structure de la table `categories`
--

CREATE TABLE `categories` (
  `id` int NOT NULL,
  `name` varchar(100) CHARACTER SET utf8mb4 COLLATE utf8mb4_unicode_ci NOT NULL,
  `description` text CHARACTER SET utf8mb4 COLLATE utf8mb4_unicode_ci NOT NULL
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_unicode_ci;
```

```
-- Structure de la table `orders`
--
CREATE TABLE `orders` (
  `id` int NOT NULL,
  `date` date NOT NULL,
  `status` enum('En attente', 'Validée', 'Annulée') CHARACTER SET utf8mb4 COLLATE utf8mb4_unicode_ci NOT NULL,
  `seats` enum('1', '2') CHARACTER SET utf8mb4 COLLATE utf8mb4_unicode_ci NOT NULL DEFAULT '1',
  `places_id` int DEFAULT NULL,
  `events_id` int NOT NULL,
  `users_id` int NOT NULL
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_unicode_ci;
```

```
-- Structure de la table `payments`
--
CREATE TABLE `payments` (
  `id` int NOT NULL,
  `date` date NOT NULL,
  `method` enum('cash', 'cheque', 'virement') CHARACTER SET utf8mb4 COLLATE utf8mb4_unicode_ci NOT NULL,
  `status` enum('en_cours', 'valide', 'annule') CHARACTER SET utf8mb4 COLLATE utf8mb4_unicode_ci NOT NULL,
  `orders_id` int NOT NULL
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_unicode_ci;
```

```
-- Structure de la table `places`
--
CREATE TABLE `places` (
  `id` int NOT NULL,
  `number` tinyint NOT NULL,
  `stands` enum('nord', 'sud', 'est', 'ouest') CHARACTER SET utf8mb4 COLLATE utf8mb4_unicode_ci NOT NULL,
  `level` enum('haut', 'milieu', 'bas') CHARACTER SET utf8mb4 COLLATE utf8mb4_unicode_ci NOT NULL
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_unicode_ci;
```

```
-- Structure de la table `athlets`
--
CREATE TABLE `athlets` (
  `id` int NOT NULL,
  `name` varchar(100) CHARACTER SET utf8mb4 COLLATE utf8mb4_unicode_ci NOT NULL,
  `first_name` varchar(100) CHARACTER SET utf8mb4 COLLATE utf8mb4_unicode_ci NOT NULL
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_unicode_ci;
```

```
-- Structure de la table `notation`
--
CREATE TABLE `notation` (
  `mvp_player` int DEFAULT NULL,
  `event_notation` enum('A', 'AA', 'AAA') CHARACTER SET utf8mb4 COLLATE utf8mb4_unicode_ci NOT NULL,
  `events_id` int NOT NULL,
  `users_id` int NOT NULL
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_unicode_ci;
```

```
--
-- Structure de la table `contacts`
--

CREATE TABLE `contacts` (
  `id` int NOT NULL,
  `users_id` int DEFAULT NULL,
  `name` varchar(100) NOT NULL,
  `first_name` varchar(100) NOT NULL,
  `email` varchar(100) NOT NULL,
  `subject` varchar(150) DEFAULT NULL,
  `message` text NOT NULL,
  `is_read` tinyint(1) NOT NULL DEFAULT '0',
  `date` date NOT NULL
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_0900_ai_ci;
```

Figure 4.3 — Scripts création des tables SQL

Remarque sur les contraintes d'intégrité référentielle :

Comme détaillé ci-dessus, **orders -> events**, **orders -> users** et **events -> categories** sont contraintes en RESTRICT au niveau du SGBD. Les relations **notation -> events/users/athlets** (CASCADE) et **payments -> orders** bénéficient elles aussi d'une contrainte FK déclarée en base, illustrée en Figure 4.4 pour la table **notation**.

Contraintes de clé étrangère		Contrainte de clé étrangère (INNODB)			
Actions	Propriétés de la contrainte	Colonne	Base de données	Table	Colonne
Supprimer	ON DELETE: CASCADE ON UPDATE: CASCADE	users_id	ACA	users	id
Supprimer	ON DELETE: CASCADE ON UPDATE: CASCADE	events_id	ACA	events	id
Supprimer	ON DELETE: CASCADE ON UPDATE: CASCADE	mvp_player	ACA	athlets	id
Nom de la contrainte			ACA		
	ON DELETE: RESTRICT ON UPDATE: RESTRICT				

Figure 4.4 — Contraintes de clé étrangère réelles sur la table notation (vue Relations, PhpMyAdmin)

Remarque sur payments, places et athlets :

Ces trois tables constituent les fondations de deux fonctionnalités volontairement mises hors du périmètre de livraison initial, mais dont la structure de données a été anticipée dès la conception : payments et places pour le futur module de gestion des présences par QR code (lorsque l'administrateur valide une réservation, ce qui correspond à l'enregistrement du paiement hors ligne, un QR code est généré côté adhérent pour lui donner accès à sa place le jour de l'événement) ; athlètes pour le vote MVP, en complément du champ notation.event_notation qui porte la note globale de l'événement.

Remarque sur Contact :

Le champ users_id est nullable pour permettre à un visiteur non connecté d'envoyer un message (cas prévu par le cahier des charges, qui distingue les échanges entre supporters et organisateurs). Le champ subject est facultatif ; is_read est un booléen plutôt qu'un enum, ce type de champ ne portant que 2 états possibles. Aucune contrainte de conservation n'étant exigée par le cahier des charges pour les messages de contact, leur suppression est un DELETE définitif, sans statut intermédiaire.

Choix de conception notables :

- orders.seats est un enum('1','2') plutôt qu'un int : traduit directement en base la règle de gestion du cahier des charges (maximum 2 places par réservation), en complément du contrôle applicatif effectué à l'insertion.
- events.scene est un enum listant les 4 tribunes + le Club House, plutôt qu'une table de référence séparée : simplifie le modèle, la liste des scènes étant fixe.
- users.status (Fermé/Privé/Public) porte directement l'exigence du cahier des charges sur la visibilité du profil d'un adhérent auprès des autres membres.
- users.profil (administrateur/service/abonne) centralise les 3 rôles dans une seule table, plutôt que 3 tables distinctes — cohérent avec le choix applicatif de mutualiser les fichiers PHP communs aux 3 profils lorsque la logique métier ne diffère pas (voir 4.2).

4.2 - Extraits de code – Composants métier

Exemple 1 :

Authentification et sécurisation du mot de passe — extrait de *login.php* :

```
<?php
session_start();
$pdo = require_once('../includes/bdd.php');
$error = "";
//Si déjà connecté
if (isset($_SESSION['email'])) {
    header('location: index.php');
    exit;
}
//Traitement POST
```

```

if ($_SERVER['REQUEST_METHOD'] === 'POST') {
    $email = $_POST['email'] ?? '';
    $password = $_POST['password'] ?? '';

    $query = $pdo->prepare("SELECT * FROM users WHERE email = ?");
    $query->execute([$email]);
    $count = $query->fetch(PDO::FETCH_ASSOC);

    if (!$count) {
        $error = "Identifiant ou mot de passe incorrect";
    } else {
        $passwordHash = $count['password'];
        if (password_verify($password, $passwordHash)) {
            $_SESSION['id'] = $count['id'];
            $_SESSION['name'] = $count['name'];
            $_SESSION['first_name'] = $count['first_name'];
            $_SESSION['email'] = $count['email'];
            $_SESSION['profil'] = $count['profil'];
            // Redirection selon profil
            if ($count['profil'] === 'administrateur' || $count['profil'] ===
'service') {
                header("Location: staff-events.php");
            } elseif ($count['profil'] === 'abonne') {
                header("Location: user-events.php");
            } else {
                header("Location: index.php");
            }
            exit;
        } else {
            $error = "Identifiant ou mot de passe incorrect.";
        }
    }
}
?>

```

Le même message d'erreur générique est renvoyé que l'email soit inconnu ou le mot de passe erroné, afin de ne pas donner d'indice à un attaquant sur l'existence d'un compte.

Exemple 2 :

Contrôle d'accès par rôle (liste blanche) — extrait commun aux pages *staff-events.php* / *staff-reservations.php* :

```

<?php
session_start();
$pdo = require_once('../includes/bdd.php');

```

```

if (!isset($_SESSION['id']) || !in_array($_SESSION['profil'], ['administrateur',
'service'])) {
    header('Location: login.php');
    exit;
}

```

Ce contrôle illustre un choix de conception délibéré : une approche de liste blanche (seuls les rôles explicitement listés passent) plutôt qu'une liste noire. Elle garantit qu'un profil futur, ajouté sans y repenser, n'hérite d'aucun accès par défaut.

Exemple 3 :

Mutualisation de fichiers entre profils — extrait de *account.php* (gestion du compte, commune aux 3 profils) :

```

<?php
session_start();
$pdo = require_once('../includes/bdd.php');

if (!isset($_SESSION['id'])) {
    header('Location: login.php');
    exit;
}

$id = $_SESSION['id'];
$profil = $_SESSION['profil'];
$error = null;

if ($profil === 'administrateur' || $profil === 'service') {
    $header = 'Location: staff-events.php';
} else {
    $header = 'Location: user-events.php';
}

...

if (empty($error)) {
    try {
        // BLOC UPDATE + REQUETTE PDO.
        header($header);
        exit;
    } catch (PDOException $e) {
        $error = "Erreur lors de la mise à jour des données, veuillez réessayer
svp.";
    }
}

```

La redirection post-modification est calculée dynamiquement selon le profil de l'utilisateur connecté, permettant à un seul fichier de servir les 3 cas.

Exemple 4 :

Application d'une règle de gestion métier : la réservation — extrait de *reservation.php* :

```
if ($seats_free > 0) {
    if ($_SERVER['REQUEST_METHOD'] === "POST") {
        $seats = $_POST['seats'] ?? '';
        try {
            if ($seats) {
                if ((int)$seats_booked + $_POST['seats'] > 2) {
                    $error = "Vous avez atteint le nombre maximum de
réservations possible";
                }
                if ($seats > $seats_free) {
                    $error = "Désolé, il n'y a pas assez de places disponibles";
                }
                if (empty($error)) {
                    $query = $pdo->prepare("INSERT INTO orders (date, status,
seats, events_id, users_id) VALUES (?, ?, ?, ?, ?)");
                    $query->execute([date('Y-m-d'), 'En attente', $seats, $id,
$_SESSION['id']]);

                    if ($_SESSION['profil'] === 'administrateur' ||
$_SESSION['profil'] === 'service') {
                        // Redirection selon profil...
                    } catch (PDOException $e) {
                        $error = "Erreur lors de la réservation, veuillez contacter le
service réservation";
                    }
                }
            }
        }
    }
}
```

\$seats_taken est calculé juste avant à partir du total déjà réservé par l'utilisateur sur cet événement. L'insertion en base n'a lieu que si aucune erreur n'a été détectée — un pattern appliqué de façon homogène sur l'ensemble des formulaires du projet.

Exemple 5 :

Changement de mot de passe : vérification en cascade — extrait de *password.php* :

```
if ($_SERVER['REQUEST_METHOD'] === 'POST') {
    $old_password = $_POST['old_password'] ?? '';
    $new_password = $_POST['new_password'] ?? '';
    $pwd_confirm = $_POST['pwd_confirm'] ?? '';

    $query = $pdo->prepare("SELECT password FROM users WHERE id = ?");
    $query->execute([$id]);
}
```

```

$currentHash = $query->fetchColumn();

if (!password_verify($old_password, $currentHash)) {
    $error = "Ancien mot de passe incorrect, veuillez saisir le bon svp";
} else {
    if (strlen($new_password) < 8 || !preg_match('/[#@!?\$%&]/', $new_password)
    || !preg_match('/[0-9]/', $new_password) || !preg_match('/[A-Z]/', $new_password)) {
        $error = "Nouveau mot de passe ne respecte pas les règles ci-dessus ";
    } else {
        if ($new_password !== $pwd_confirm) {
            $error = "Les nouveaux mots de passe ne sont pas identiques";
        } else {
            $newHash = password_hash($new_password, PASSWORD_DEFAULT);
            $currentHash = $newHash;
        }
    }
}
}

```

Cette structure en cascade (plutôt qu'une suite de "if" indépendants) garantit qu'aucune vérification ultérieure ne peut écraser silencieusement une erreur déjà détectée.

4.3 - Extraits de code – Composants d'accès aux données

L'ensemble des accès à la base de données du projet passe par des requêtes préparées PDO, sans exception. Cette section présente les 4 opérations CRUD de base ainsi qu'un cas de calcul agrégé.

Select : (vérification des places déjà réservées) — extrait de *reservation.php* :

```

$query = $pdo->prepare("SELECT SUM(seats)
    FROM orders
    WHERE events_id = ?
    AND users_id = ?
    AND status NOT IN ('Annulé')
    ");
$query->execute([$id, $_SESSION['id']]);
$seats_booked = $query->fetchColumn();

if ($seats_booked === null) {
    $seats_booked = 0;
}

$seats_free = $event['capacity'] - $seats_taken;

```

Cette requête illustre un point d'attention rencontré en cours de développement : une première version filtrait sur orders.id en plus de events_id/users_id, ce qui limitait le calcul à une seule ligne au lieu de la somme réelle des réservations de l'utilisateur sur l'événement. La correction a consisté à ne conserver que les critères events_id et users_id dans la clause WHERE.

Insert : (nouvelle réservation) :

```
if (empty($error)) {
    $query = $pdo->prepare("INSERT INTO orders (date, status, seats, events_id,
        users_id) VALUES (?, ?, ?, ?, ?)");
    $query->execute([date('Y-m-d'), 'En attente', $seats, $id, $_SESSION['id']]);
    // Redirection selon profil...
```

Update : (changement de mot de passe) — extrait de *password.php* :

```
if (empty($error)) {
    try {
        $query = $pdo->prepare(
            "UPDATE users SET
            password = :password
            WHERE id = :id"
        );
        $query->execute([
            ':password' => $currentHash,
            ':id' => $id
        ]);
    }
    // Redirection selon profil...
```

Ici, la syntaxe à paramètres nommés (:password, :id) est utilisée plutôt que la syntaxe positionnelle (?) employée ailleurs dans le projet (voir select et Insert ci-dessus) C'est un choix de lisibilité ponctuel plutôt qu'une règle systématique appliquée partout.

Delete : (événement) — extrait de *staff-delete-event.php* :

```
try {
    $query = $pdo->prepare("DELETE FROM events WHERE id = ?");
    $query->execute([$id]);
} catch (PDOException $e) {
    $_SESSION['error'] = "Erreur : Impossible de supprimer cet événement, veuillez réessayer svp";
}
```

Requête avec agrégat et filtre d'exclusion : extrait de *reservation.php*, pour le calcul des places encore disponibles sur un événement :

```
$query = $pdo->prepare("SELECT SUM(seats)
    FROM orders
    WHERE events_id = ?
    AND status NOT IN ('Annulé')
    ");
$query->execute([$id]);
$seats_taken = $query->fetchColumn();
```

La clause status NOT IN ('Annulé') garantit qu'une réservation annulée libère bien sa place dans le calcul de disponibilité, sans qu'il soit nécessaire de supprimer physiquement la ligne correspondante — ce qui préserve un historique complet des réservations, y compris annulées.

5. Les éléments de sécurité de l'application

La sécurité de MNS FC a été traitée comme une contrainte transversale, intégrée dès la conception de chaque page plutôt qu'ajoutée après coup. Le principe appliqué tout au long du projet : toute donnée saisie par un utilisateur est considérée comme non fiable, intentionnellement ou pas erreur, aussi bien par l'adhérent que par l'administrateur, aussi bien côté client que côté serveur.

5.1 Protection contre les injections SQL

L'intégralité des requêtes SQL de l'application utilise des requêtes préparées PDO, sans aucune exception. Aucune variable n'est concaténée directement dans une chaîne SQL, éliminant par construction tout risque d'injection SQL : la structure de la requête et les données sont transmises séparément au serveur MySQL.

5.2 Protection contre les failles XSS (Cross-Site Scripting)

Toute donnée provenant de l'utilisateur et réaffichée dans une page HTML passe systématiquement par *htmlspecialchars()* avant affichage. Cette règle a été appliquée avec la même rigueur même à des données que le développeur maîtrise lui-même (messages système), par principe de défense en profondeur.

5.3 Sécurisation des mots de passe

Afin de suivre les standards en termes de sécurité, les mots de passe ne sont jamais stockés ni manipulés en clair. La création repose sur *password_hash()* (hash à sens unique, sel aléatoire intégré, différent à chaque appel même pour un mot de passe identique) et la vérification sur *password_verify()* (extrait le sel du hash stocké et recalcule la comparaison en interne, sans jamais exposer le hash à une comparaison directe).

Une règle de complexité est imposée à la création et à la modification du mot de passe (8 caractères minimum, au moins une majuscule, un caractère spécial, un chiffre), contrôlée à la fois côté client (JavaScript, retour immédiat) et côté serveur (PHP, avec *preg_match()*).

Retour d'expérience notable : la vérification de complexité côté serveur était initialement absente sur la page de changement de mot de passe — seul un affichage visuel (JavaScript) indiquait à l'utilisateur si son mot de passe respectait les critères, sans réellement bloquer l'enregistrement d'un mot de passe faible. J'ai identifié ce point lors d'une relecture du code, à la suite de cela j'ai ajouté ce point à la phase des tests, puis une correction en ajoutant le contrôle serveur. Ce cas illustre concrètement pourquoi la validation côté client ne peut jamais constituer, à elle seule, une mesure de sécurité.

5.4 Gestion des sessions et contrôle des rôles

L'authentification repose sur les sessions PHP natives. Chaque page nécessitant une authentification vérifie la présence d'une session valide avant tout traitement. Les pages réservées aux profils administrateurs et service réservation appliquent en plus un contrôle

explicite du rôle, avec une approche de liste blanche (seuls les rôles explicitement autorisés passent, tout profil inconnu ou futur est bloqué par défaut) plutôt qu'une liste noire.

5.5 Validation des données en entrée côté serveur

Chaque formulaire dispose d'une double validation : côté client (JavaScript, retour immédiat sur le format attendu — email, téléphone, code postal, mots de passe) et côté serveur (PHP, contrôle final et non contournable — *filter_var()*, *preg_match()*, contrôle de présence des champs obligatoires). Cette redondance volontaire illustre le principe retenu sur l'ensemble du projet : le JavaScript améliore l'expérience utilisateur (contre l'oubli ou l'incompréhension), PHP garantit la sécurité réelle.

5.6 Exemple représentatif : la règle métier des 2 places maximum par réservation

Le cahier des charges impose qu'un adhérent ne puisse pas réserver plus de 2 places par évènement. Cette règle illustre bien l'articulation des deux couches de contrôle : côté client, le <select> du formulaire de réservation limite dynamiquement les options proposées selon ce que l'utilisateur a déjà réservé ; côté serveur, avant tout enregistrement, une requête recalcule le total déjà réservé et bloque l'insertion si le total dépasserait 2. Sans ce contrôle serveur, un utilisateur connaissant les outils de développement de son navigateur pourrait contourner le <select> et poster une valeur non autorisée directement au serveur.

6. Les jeux d'essai :

Afin de garantir la qualité, la conformité fonctionnelle et la sécurité de l'application Gest Club, une série de recettes a été menée avec les jeux d'essai suivants :

- Les cas nominaux (utilisation classique conforme aux attentes).
- Les cas limites et d'erreurs (champs vides, saisies incorrectes, dépassement de quotas).
- Les tests de sécurité (contrôle d'accès par rôle, étanchéité des formulaires, règles de mot de passe).

L'ensemble des tests a été exécuté sur l'environnement de recette (Docker local) puis validé sur l'hébergement de production (Hostinger).

6.1 Tests de sécurité et d'authentification

Ref	Règle métier	Données d'entrée	Résultat attendu	Résultat obtenu
T07-10	Identifiant invalide	Email inconnu	Identifiant ou mot de passe incorrect	OK
T07-13	Mauvais mot de passe	MDP random	Identifiant ou mot de passe incorrect	OK
T09-16	Complexité du MDP	MDP simple	Le mot de passe ne respecte pas les règles ci-dessus	KO
T09-18	Complexité du MDP	MDP simple	Le mot de passe ne respecte pas les règles ci-dessus	OK
T07-21	Accès direct par URL	Url modifiée	Redirection vers index.php	OK

Tableau 6.1 - Les exemples de tests de sécurité et authentification.

Analyse :

Le premier test (T09-16) a révélé l'absence de contrôle serveur sur la complexité du nouveau mot de passe lors de son changement : seul un affichage visuel JavaScript indiquait à l'utilisateur si son mot de passe respectait les critères, sans réellement bloquer l'enregistrement d'un mot de passe faible côté serveur. La correction a consisté à ajouter, dans password.php, une vérification preg_match() sur la longueur et les critères de complexité avant tout enregistrement. Le test rejoué (T09-18) a confirmé le blocage effectif d'un mot de passe non conforme.

6.2 Tests des règles métier

Ref	Règle métier	Données d'entrée	Résultat attendu	Résultat obtenu
T08-10	Règle des 2 places max	Un user ayant déjà 1 place tente de réserver 2 supp.	Vous avez atteint le nombre max de réservations possible	OK
T08-13	Disponibilité des places	Réservation pour un événement	Les places réservées sont déduites du total disponible	OK
T08-17	Disponibilité des places	Réservation puis annulation d'une commande	La place annulée est réintégrée dans la jauge des places disponibles	KO
T08-18	Disponibilité des places	Réservation puis annulation d'une commande	La place annulée est réintégrée dans la jauge des places disponibles	OK
T08-22	Changement de profil	Modification du profil depuis account.php	Un admin ne doit pas pouvoir changer son propre profil afin de garantir d'en avoir tjrs un minimum	KO

Tableau 6.2 - Les exemples de tests des règles métier.

Analyse :

Le premier test (T08-17) a montré que le nombre de places disponibles affiché sur la page événement restait inchangé après l'annulation d'une réservation. L'analyse a révélé une erreur de variable dans la boucle de calcul : le code utilisait `$event['seats']` (une variable hors de portée dans ce contexte) au lieu de `$row['seats']` (la valeur réelle de la ligne en cours de traitement). La correction de cette variable a rétabli un calcul correct de la jauge de disponibilité, confirmé par le test rejoué (T08-18).

Le test T08-22 a révélé qu'un administrateur pouvait initialement modifier son propre champ profil depuis le formulaire *account.php*, avec un risque réel de rétrogradation accidentelle (un admin qui se repasse en service ou abonne, sans qu'aucun autre administrateur ne puisse reprendre la main). La correction a consisté à retirer entièrement le champ de sélection du profil du formulaire *account.php* : le profil d'un utilisateur n'est modifiable que via la gestion des adhérents (*admin-modify-user.php*), réservée à l'administrateur, jamais depuis sa propre page de compte.

6.3 Tests d'intégration et RGPD / Formulaires

Ref	Règle métier	Données d'entrée	Résultat attendu	Résultat obtenu
T09-20	Robustesse GET/POST	Requête de calcul des places réservées, appelée en GET puis en POST	Calcul correct sans avertissement PHP	OK
T09-27	Envoi de message via contact	Visiteur non connecté	Champ <code>users_id</code> doit être Null	OK

T09-28	Envoi de message via contact	Visiteur connecté	Champ users_id renseigné	OK
T09-21	Calcul du total de réservations d'un adhérent	Somme des réservations actives (hors annulées) d'un adhérent sur un événement	Somme correcte, limitée aux réservations de l'adhérent connecté	OK

Tableau 6.3 - Les exemples de tests d'intégration.

Analyse :

Le test T09-20 a permis de détecter un avertissement PHP sur *reservation.php* : le calcul des places déjà réservées utilisait `$_GET['id']` en dur, qui ne reflétait plus l'identifiant de l'événement lorsque la page était appelée en POST (soumission du formulaire). La correction a consisté à utiliser la variable `$id` (déjà calculée correctement pour les deux méthodes GET et POST) dans cette requête.

Le test T09-21 a par ailleurs révélé une seconde erreur dans le calcul du total déjà réservé par un adhérent : la requête filtrait initialement sur `orders.id` en plus de `events_id` et `users_id`, limitant le résultat à une seule ligne au lieu de la somme réelle des réservations de l'adhérent sur l'événement. Le retrait de ce critère erroné a corrigé le calcul.

7. La veille sécurité

La veille sur les vulnérabilités de sécurité web a été menée tout au long du projet, principalement via le référentiel OWASP (Open Web Application Security Project), complété par la documentation MDN et un suivi de comptes techniques spécialisés (Flipboard, influenceurs tech). Plutôt que de tenter de couvrir exhaustivement les 10 catégories de l'OWASP Top 10, la veille a été volontairement ciblée sur les 4 familles de vulnérabilités les plus pertinentes au regard des fonctionnalités réellement développées sur MNS FC.

7.1 Injection SQL

Un attaquant glisse du code SQL dans un champ de saisie pour manipuler ou extraire des données de la base. Cette vulnérabilité a été neutralisée par construction : l'intégralité des requêtes SQL du projet utilise des requêtes préparées PDO, sans aucune exception ni concaténation directe de variable dans une chaîne SQL (voir section 5.1).

7.2 Broken Authentication (gestion défailante de l'authentification)

Regroupe les mauvaises pratiques de gestion des mots de passe et des sessions : stockage en clair, absence de règle de complexité, sessions mal invalidées. Le projet s'appuie sur *password_hash()* / *password_verify()*, une règle de complexité imposée côté serveur, et les sessions PHP natives systématiquement vérifiées avant tout traitement sensible (voir sections 5.3 et 5.4).

7.3 Broken Access Control (contrôle d'accès défaillant)

Concerne les cas où un utilisateur accède à des pages ou actions qui ne devraient pas lui être accessibles selon son rôle. Le projet applique une vérification de rôle en liste blanche sur chaque page sensible (administrateur/service réservation), garantissant qu'un profil non explicitement autorisé, y compris un profil futur non encore anticipé, soit bloqué par défaut plutôt qu'autorisé par défaut (voir section 5.4).

7.4 Cross-Site Scripting (XSS)

Un attaquant injecte du code HTML ou JavaScript malveillant qui s'exécute dans le navigateur d'un autre utilisateur, via un champ de saisie non filtré à l'affichage. Le projet applique *htmlspecialchars()* de façon systématique sur tout affichage de donnée provenant de l'utilisateur, y compris sur des messages générés par l'application elle-même, par principe de défense en profondeur (voir section 5.2).

Cette veille ciblée a directement permis d'identifier, en cours de développement, la faille corrigée sur la complexité du mot de passe (voir section 5.3 et test T09-16 en section 6.1) : un exemple concret où la veille théorique a débouché sur une correction réelle du code, plutôt que de rester un exercice de style déconnecté du projet.

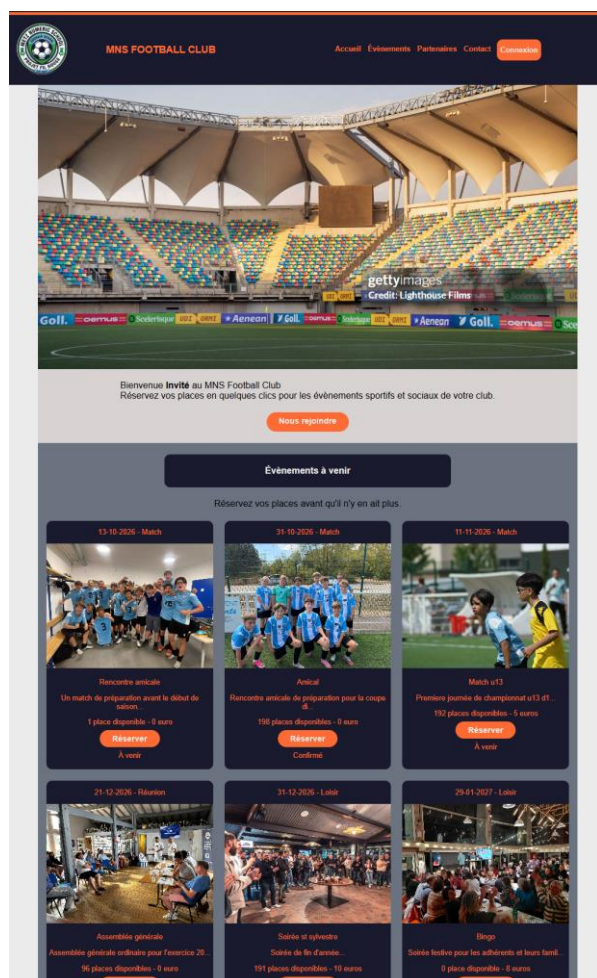
8. Annexes

8.1 Les maquettes des interfaces utilisateur



Tableau 8.1 - Exemples de maquettes des modales.

8.2 Les interfaces utilisateur et le code correspondant



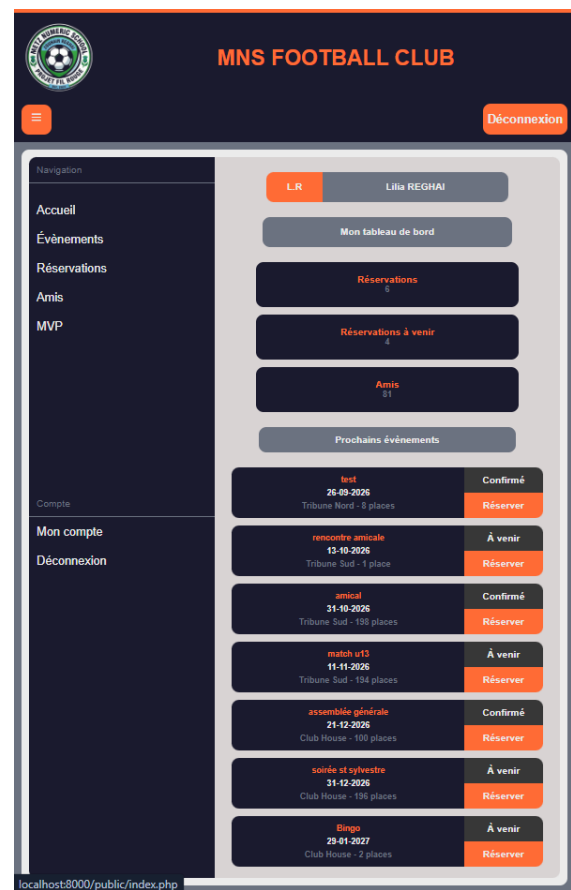
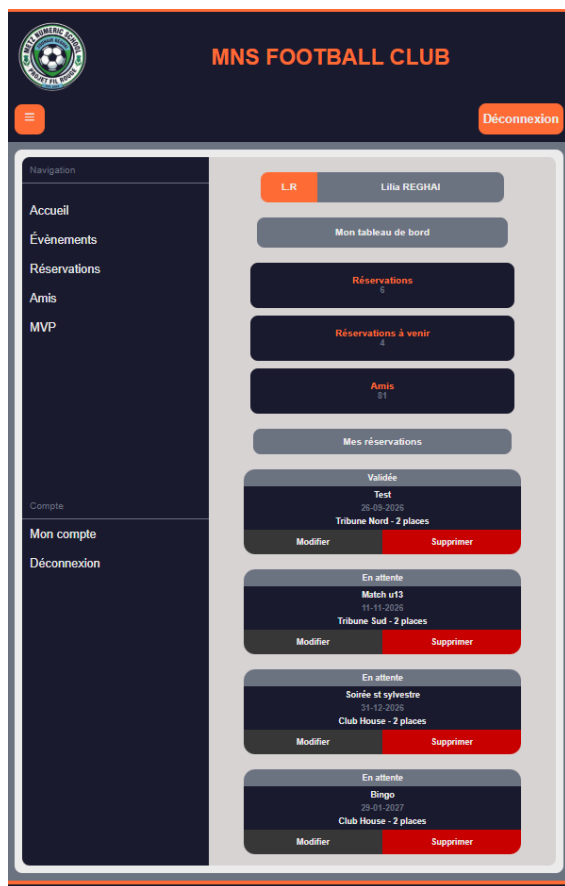


Figure 8.2 - Exemples de pages utilisateur en format tablette.

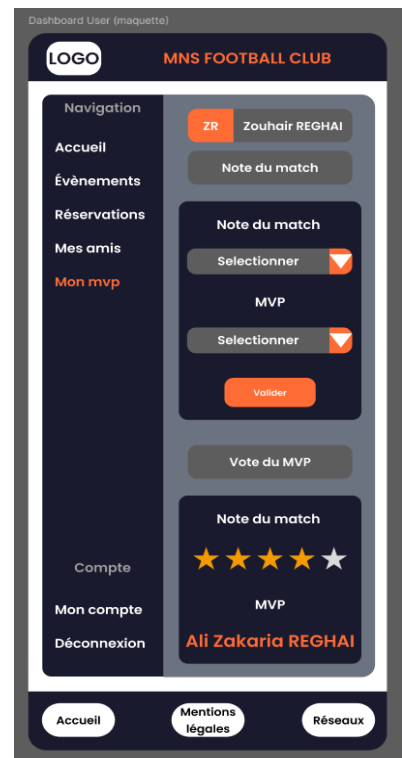
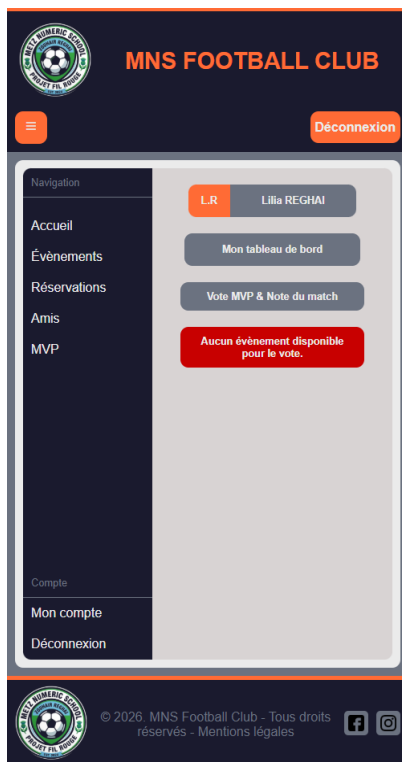


Figure 8.3 - Exemples maquettes et pages système de vote en format mobile.

8.3 Le code des composants métier les plus significatifs

Le fichier modal.js intercepte la soumission des formulaires de suppression pour afficher une modale de confirmation avant d'exécuter réellement l'action (voir 3.6)

```
//Traitement modale information (succès ou erreur après action)
const modalConfirm = document.querySelector('.modal-confirm');
const Icon = document.querySelector('.modal-icon');
const p = document.querySelector('.modal-confirm p');
const button = document.querySelector('.modal-confirm button');

if (modalMessage !== null) {
  p.innerText = modalMessage;
  Icon.innerHTML = modalIcon;
  modalConfirm.classList.add('is-open');
}

button.addEventListener('click', (e) => {
  modalConfirm.classList.remove('is-open');
})

//Traitement modale confirmation (avant suppression)
const forms = document.querySelectorAll('.form-delete');
const modalAlert = document.querySelector('.modal-alert');
const confirmButton = modalAlert.querySelector('.modal-cta button');
const cancelButton = modalAlert.querySelector('#cancel');
let currentForm = null;

forms.forEach((form) => {
  form.addEventListener('submit', (event) => {
    event.preventDefault();
    modalAlert.classList.add('is-open');
    currentForm = form;
  })
})

confirmButton.addEventListener('click', () => {
  currentForm.submit();
  modalAlert.classList.remove('is-open');
})

cancelButton.addEventListener('click', () => {
  modalAlert.classList.remove('is-open');
})
```

*Figure 8.4 - Extrait de code modal.js
(Interception formulaire, confirmation avant action)*

8.4 Le code de composants d'accès aux données les plus significatifs

Voir section 4.3 pour les extraits SQL représentatifs (SELECT, INSERT, UPDATE, DELETE, requête d'agrégat).